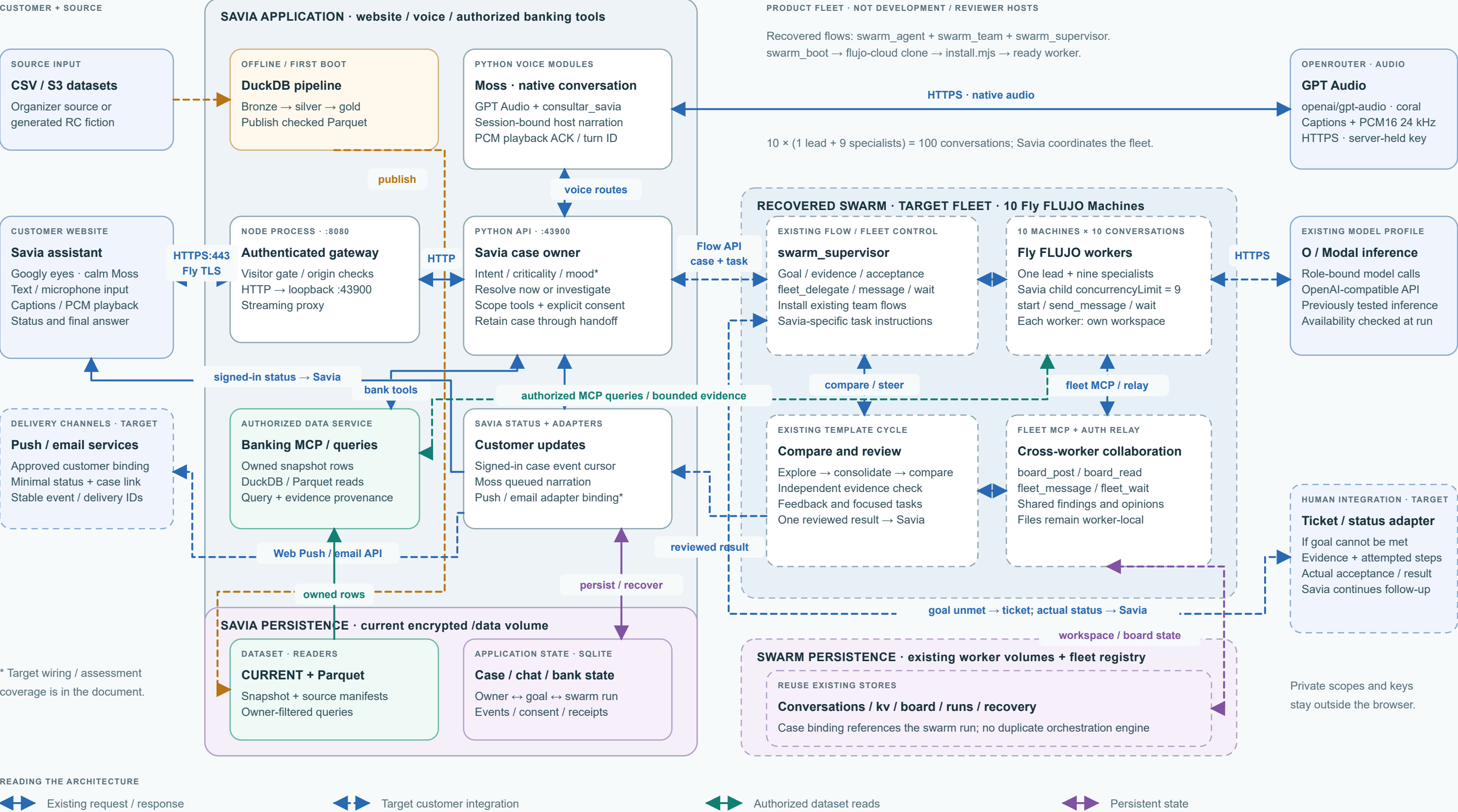


Savia · technical product architecture

One customer assistant · recovered swarm template · target: 10 Fly FLUJO workers × 10 agent conversations = 100 collaborating AIs



Savia — technical product architecture

Design updated 5 October 2026. Savia is the customer's assistant throughout the life of a problem. It resolves a request immediately when possible; otherwise it commissions a team of 100 specialist AI agents, reviews their findings and returns one coherent answer. An unresolved goal becomes a human ticket. Status reaches the customer through Savia, browser push or email.

This is the owner-directed product architecture. The target uses the recovered Claude swarm implementation: ten Fly FLUJO Machines, each with ten intercommunicating conversations. Savia's customer application coordinates that fleet. Development and reviewer Machines are outside the product. Section 7 maps the design to current evidence.

1. Runtime, network and data boundaries

Boundary	Interface / execution	Responsibility
Customer browser	React/Vite website; HTTPS; Web Audio	Savia dialog, googly eyes, calm Moss voice, text, selected account context and visible progress
Submission ingress	Fly edge TLS → Node gateway 0.0.0.0:8080	Visitor gate, exact host/origin checks and streaming proxy
Savia application	Python/Uvicorn 127.0.0.1:43900 ; gateway's fixed loopback upstream	Customer session, request assessment, authorized bank tools, voice and case projection
Orchestration foundation	Existing FLUJO chat / flow / MCP / workspace / recovery interfaces	Execute the case workflow, bind approved models/tools, schedule and recover work
Swarm template	Recovered swarm_supervisor , swarm_team , swarm_agent ; swarm_boot for cloning	Explicit roles/tasks, model/tool bindings, collaboration and review structure
Case coordinator	Savia's application-owned flow and policy on that foundation	Define the goal, commission the team, accept reviewed findings, escalate and follow through
Agent team	10 Fly FLUJO Machines × (one team lead + nine specialist conversations)	100 team conversations; investigate, compare findings, challenge conclusions and share evidence
Data plane	Offline CSV/S3 preparation → DuckDB → published Parquet; scoped MCP tools	Preserve snapshot/query lineage and return authorized evidence
Human integration	Existing ticketing/status adapter over authenticated API	Create/recover a ticket, expose actual acceptance and return a human result
Delivery integrations	In-portal API, existing Web Push and email adapters	Deliver meaningful changes through customer-approved channels

The design reuses the recovered `swarm-teams/template/flows.mjs` and its installer. `install.mjs` adds the model row, fleet MCP entry and flows to a FLUJO workspace; FlowSpecs compile through FLUJO's API. `flujo-cloud` clones the tool-free `swarm_boot` workspace, then installs the tool/team flows on the ready worker. Savia specializes task instructions and permitted tools; a new template framework is unnecessary. FLUJO stays generic; banking policy and authority remain in this application and Banking MCP.

Current ingress and application ports above are source-verified. Target swarm, ticket and notification connections are dashed in the diagram. Their exact installed bindings must be supplied by the ecosystem owners; a proposed connection is not a claim of a deployed integration.

2. Case lifecycle and request assessment

The authenticated host binds a case to the customer and a concrete goal. Assessment extracts intent, criticality and mood from the conversation, recording uncertainty and missing information. Criticality controls priority and escalation; intent selects the permitted workflow; mood controls wording and pace. None of these signals grants banking authority or proves resolution.

State / transition	Required evidence or decision	Customer-visible result
assessing	Intent, criticality, mood, missing information and permitted data scope	Savia acknowledges the problem and asks for needed clarification
Immediate answer → awaiting_customer	Verified answer satisfies the stated goal	One explanation with evidence and remaining limits
Investigation → investigating	Immediate authorized lookup cannot accomplish the goal	Team template is instantiated with a case plan; Savia retains the problem
Team result → reviewing	Findings contain provenance and explicit unanswered questions	Results and opinions are compared; conflicts trigger focused follow-up tasks
Reviewed synthesis → awaiting_customer	Findings satisfy acceptance criteria and withstand review	Savia returns one combined result
Goal unmet → human_pending	Deadline/budget exhausted, high criticality, requested human help or unresolved disagreement	One idempotent human ticket, with actual reference/status
Human acceptance → human_working	Ticket integration confirms assignment/acceptance	Savia reports actual human progress
Goal accomplished → resolved	Goal-specific verification; customer confirmation when required	Recorded outcome and final explanation; investigation stops

The case survives logout, a new chat and a runtime restart. Recovery uses the existing workspace/run records and the case-to-run binding. Savia continues status and follow-up while a human owns the ticket. Informational closure, ticket submission and a verified bank outcome remain distinct events.

3. Existing swarm template and ten-by-ten collaboration

The recovered template already splits a goal, staffs different approaches, consolidates findings, compares measured results and assigns independent checks. Savia supplies case-specific tasks such as transaction chronology, amount/date reconciliation, aggregate patterns, policy interpretation and alternate hypotheses. Use these existing flows and collaboration tools while the customer continues speaking to Savia:

Existing mechanism	Concrete interface / use
<code>swarm_supervisor</code>	Root goal and evidence-driven acceptance; read <code>fleet_info</code> and <code>kv_get("swarm_state")</code> before starting new work
<code>swarm_team</code>	One worker's lead; explore, consolidate, compare, build, pick, then independent claim check
<code>swarm_agent</code>	One specialist conversation receives its explicit task/angle, posts findings and reports evidence
Within-worker collaboration	Native <code>start_subflow_*</code> , <code>subflow_send_message</code> , <code>subflow_wait</code> , <code>subflow_list</code>
Across-worker coordination	Fleet MCP <code>fleet_delegate</code> , <code>fleet_message</code> , <code>fleet_wait</code> ; authenticated relay connects remote workers to the existing controller
Shared findings	<code>board_post</code> / <code>board_read</code> across workers; worker files remain local, so share provenance/results or a reproducible data recipe
Recovery	Existing conversation/run records, fleet registry and <code>swarm_state</code> key-value notes; reuse active run identity and respect uncertain cleanup/replay state

The stock `swarm_team` spec has `concurrencyLimit=10` for child agents in addition to its lead. For the requested literal ten conversations per Machine, specialize it to one lead plus nine child specialists (`concurrencyLimit=9`). Ten Machines therefore contribute 100 team conversations; Savia's root coordination remains separate. Record every lead/child conversation ID when qualifying this count.

Template / case contract	Required fields or rule
Template identity	Version, source hash, role inventory and installation/deployment reference
Specialist role	Role ID, objective, explicit instructions, prompt revision, model binding and allowed tool/data scope
Collaboration structure	Task dependencies, shared-workspace projections, review assignments, feedback loops and synthesis criteria
Case instantiation	Case ID, goal, acceptance criteria, priority, deadline, permitted evidence, budget and template revision
Assignment	Task ID, specialist role, objective, dependencies, execution lease and attempt identity
Finding	Task/agent ID, bounded conclusion, evidence/query/snapshot references, confidence and open questions

Template / case contract	Required fields or rule
Peer review	Finding under review, agreement/disagreement, evidence checks and a specific requested additional test
Synthesis	Accepted findings and unresolved conflicts evaluated against the original goal; a majority vote alone is insufficient

Use the existing controller's worker/tree caps and native subflow bound to enforce that layout. Conversation count, simultaneous active execution and inference-request count must be measured separately. Per-task timeouts, total token/query/cost limits and a case deadline bound the customer goal. Recovered or retried work retains identity; held or unknown work is not blindly replayed.

Large-data tasks use authorized server-side DuckDB queries and partitioned Parquet reads. Models receive bounded results and aggregate evidence rather than full source files. Findings retain snapshot/query provenance so peers can reproduce a claim and reuse evidence. Application/tool ownership checks enforce data scope independently of prompt instructions.

4. Data, workspace and persistence

Store / boundary	Information	Owner and access
Published snapshot	CURRENT , manifests, source hashes, Parquet partitions and quarantine/rejects	Offline pipeline publishes; approved server/MCP readers query
Portal and bank stores	Profile sessions, public chat, selected context, consent and verified receipts	Authenticated Savia host and Banking MCP; models cannot sign bank actions
Case-to-swarm binding	Owner, goal, case status, template revision and run/workspace reference	Savia application; customer sees a bounded authenticated projection
Existing swarm workspace	Assignments, findings, peer review, attempts and recovery records	Existing orchestration/swarm foundation; task-scoped access
Ticket linkage	Escalation ID, ticket reference, packet hash, acceptance and human result	Ticket/status integration with verified callback or bounded poll
Notification state	Case/event cursor, recipient/channel binding, delivery identity and retry state	Existing notification integration; customer preference and authority checked
Voice memory	Active turn, registered result, pending PCM playback and bounded voice history	Current voice session; discarded at process restart

Reuse the workspace and delivery stores already provided by the ecosystem. Persist the application-owned case-to-run mapping and event projection alongside existing inquiry state; do not copy the orchestration engine's task ledger into a competing scheduler. Banking authority and receipts stay separate from swarm findings. A specialist opinion cannot authorize a bank mutation.

Organizer CSVs can enter through read-only S3 ingestion, then the checked offline DuckDB pipeline publishes a snapshot. The current RC instead generates fictional CSVs and runs the same project pipeline at first boot. These are alternative inputs to one data plane. Neither browser nor model receives direct S3 access. A data refresh must publish a new manifest before an investigation changes its evidence basis.

5. Integrated eyes, Moss voice and foreground conversation

`Eyes.tsx` renders the pair of googly eyes inside the Savia assistant dialog. `useSaviaVoice` supplies listening/thinking/speaking activity and audio level. Moss is the calm persona, using provider voice `coral`. Spanish and Brazilian Portuguese are supported. Text input can use the same case path; the choice of text-only delivery does not change the team contract.

Existing interface	Protocol / behavior
POST <code>/api/voice/turn</code>	Authenticated base64 WAV audio, text message or registered host result; native <code>openai/gpt-audio</code> over OpenRouter HTTPS
Voice response	<code>application/x-ndjson</code> : start, heard, caption, audio, delegate, complete/error; base64 PCM16 mono at 24 kHz
<code>consultar_savia</code>	Voice requests host work; browser sends it through authenticated <code>/api/chat/messages</code> with current selected context
Host-result narration	Exact host answer remains visible; server registers it to the current session for one-use narration within five minutes
POST <code>/api/voice/played</code>	Active turn ID and exact PCM sample count; matching complete playback commits spoken voice history
GET <code>/api/assistant/voice-update</code>	Case/event cursor; server-owned meaningful update enters the same voice conversation after foreground playback

Voice is presentation and conversation; the case workflow owns investigation state. A background result must not interrupt foreground speech. The existing UI polls working cases every two seconds and other cases every 15 seconds, deduplicates events and queues eligible narration. The native model receives audio; a bounded recognition pass separately gates explicit banking delegation. A process restart preserves disk-backed cases/chat but retires in-memory audio turns.

6. Human ticket and status delivery

The escalation packet contains the goal, criticality, permitted customer context, accepted findings with provenance, disagreements, attempted actions and unresolved questions. Reuse the existing ticket adapter under a stable escalation identity. Ticket creation/recovery must be idempotent. A ticket reference establishes submission; human acceptance requires a verified status event. Human findings re-enter review and Savia reports the resulting outcome.

Channel	Technical path	Condition
Logged-in customer	Authenticated case API → event cursor → UI → optional Moss narration	Meaningful new status/result; wait for foreground playback to finish

Channel	Technical path	Condition
Browser push	Existing delivery adapter → customer-bound Web Push subscription → service worker	Customer-approved subscription; minimal status plus a protected case link
Email	Existing email adapter → verified recipient	Channel preference and recipient verified; no raw account dataset in the message
Human status	Authenticated callback or bounded ticket poll → case event → delivery adapter	Verified ticket/case binding and monotonic status cursor

Deduplicate delivery by `(case_id, event_id, channel)` and reuse that identity across bounded retries. A sent update is not a resolved goal. Unchanged checks stay quiet. When the customer returns, Savia catches up from persisted case events. Push and email are channels of the same assistant, not independent agents.

7. Implementation coverage and integration gaps

Product requirement	Current evidence / remaining integration
Website, googly eyes and calm Moss	Deployed integrated Savia UI, authenticated native voice, host-result binding and playback checks
Intent, criticality and mood	Intent/emotional-context policy exists; complete structured three-signal assessment needs wiring
10 Fly × 10 conversations	Recovered template and installer exist. Specialize the native bound to nine children plus lead, bind Savia cases/tools and qualify the exact ten-by-ten communicating customer run
Large-data investigation	DuckDB/Parquet pipeline and scoped banking readers exist; expose permitted query/evidence contracts to template roles
Persistent problem ownership	Durable inquiry/events and background checks exist; bind Savia cases to the foundation's actual swarm workspace/run lifecycle
Human ticket	Local handoff packet exists; connect the real ticket/status integration and verify actual human acceptance
Logged-in and spoken status	Implemented and recorded in the integrated customer path
Push/email	Reuse intended adapters; exact installation, customer binding and delivery acceptance still need verification

The current customer RC is one Fly Machine `851d7dc4460048` in `iad`, one shared CPU, 1024 MB RAM and encrypted 1 GB volume `vol_4qlomp91ly8qn98r` at `/data`. Its ingress is 8080; Python listens on loopback 43900. Live metadata and public `/healthz` HTTP 200 were read on 5 October 2026. It contains a temporary two-role inquiry demonstrator. The recovered swarm separately proves existing worker/team mechanics; neither establishes the requested integrated ten-by-ten customer run.

The recovered README records a real Fly team with two parallel agents and a mixed tree with 18 Fly leaf sandboxes live together. These qualify different run scopes. Existing O/Modal inference is already tested; current availability and the exact ten-worker customer integration remain separate checks. This diagram does not provision Machines or replay prior work.

Frozen application source: `9d77a7599128b668b0e34f9c2937eb40b6bd3824` . Image: `sha256:484fe8edc07ac37dd78f61deb7a08254352b8ce597fa8265f917706e93d199a5` . Customer-path acceptance is recorded in the release evidence. The RC implementation reference gives exact database paths, APIs and current failure limits. The 100-agent design remains the product target.

8. Source and regeneration

Subject	Source
Current network/process setup	Fly config, gateway, supervisor, launcher
Context, policy and bank authority	Workflow, policy, host contract
Eyes, voice and updates	Eyes, voice hook, Conversation, inquiry API
Case persistence foundation	InquiryService, inquiry contract
Data plane	Pipeline, fictional preparation
Release evidence	Release record, native capture
Generic/domain boundary	FLUJO product boundary

Recovered source: `swarm-teams/template/flows.mjs` , `install.mjs` , `fleet/relay.mjs` and `README.md` in the Claude `swarm-teams` worktree of `iambrokeplshlp` . Those files were inspected directly; the flow file's SHA-256 is `bd38b40c79c2bf134fe88880470aaaba85db673ec5203b03e38e4d6bd441faf9` . Exact flow/model/tool bindings and ticket/notification destinations are integration inputs, not invented endpoints. New case-state names above specify the target lifecycle. The current RC calls OpenRouter directly; the target uses the existing FLUJO worker fleet without adding banking code to FLUJO main.

Run `python docs/architecture/build-landscapes.py` to regenerate SVG, document HTML and viewer. The September 30 diagrams remain dated historical tabs. The history Landscape distinguishes the Savia product architecture from development tooling. No development/reviewer Machine is a submission component.